

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение

высшего образования

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное
обеспечение

Направление подготовки: Программная инженерия 09.04.04

Дисциплина: Системное программное обеспечение

Практическое задание №3

Обучающийся: Пименов Глеб Андреевич 506758

Группа: Р4115

Дата 20.01.2026

Санкт-Петербург

2026

Оглавление

1. Цели	2
2. Задачи	2
3. Описание работы	7
4. Аспекты реализации	8
4.1. Компоненты системы	8
4.2. Модель регистров	8
4.3. Модель памяти	9
4.4. Инструкции	9
4.5. Алгоритм генерации листинга	11
Результаты	17
Выводы	20

1. Цели

Реализовать формирование линейного кода в терминах некоторого набора инструкций посредством анализа графа потока управления для набора подпрограмм. Полученный линейный код вывести в мнемонической форме в выходной текстовый файл

2. Задачи

Подготовка к выполнению по одному из двух сценариев:

1. Составить описание виртуальной машины с набором инструкций и моделью памяти по варианту
 - a. Изучить нотацию для записи определений целевых архитектур
 - b. Составить описание ВМ в соответствии с вариантом
 - i. Описание набор регистров и банков памяти

ii. Описать набор инструкций: для каждой инструкции задать структуру операционного

кода, содержащего описание операндов и набор операций, изменяющих состояние ВМ

1. Описать инструкции перемещения данных и загрузки констант

2. Описать инструкции арифметических и логических операций

3. Описать инструкции условной и безусловной передачи управления

4. Описать инструкции ввода-вывода с использованием скрытого регистра в

качестве порта ввода-вывода

iii. Описать набор мнемоник, соответствующих инструкциям ВМ

с. Подготовить скрипт для запуска ассемблированного листинга с использованием описания ВМ:

i. Написать тестовый листинг с использованием подготовленных мнемоник инструкций

ii. Задействовать транслятор листинга в бинарный модуль по описанию ВМ

iii. Запустить полученный бинарный модуль на исполнение и получить результат работы

iv. Убедиться в корректности функционирования всех инструкций ВМ

2. Выбрать и изучить прикладную архитектуру системы команд существующей ВМ

а. Для выбранной ВМ:

i. Должен существовать готовый эмулятор (например qemu)

ii. Должен существовать готовый тулчейн (набор инструментов разработчика):

компилятор Си, ассемблер и дизассемблер, линковщик, желательно отладчик

б. Согласовать выбор ВМ с преподавателем

- с. Изучить модель памяти и набор инструкций ВМ
- d. Научиться использовать тулчейн (собирать и запускать программы из листинга)
- е. Подготовить скрипт для запуска ассемблированного листинга с использованием эмулятора
- i. Написать тестовый листинг с использованием инструкций ВМ
- ii. Задействовать ассемблер и компоновщик из тулчейна
- iii. Запустить бинарный модуль на исполнение и получить результат его работы

Порядок выполнения:

1. Описать структуры данных, необходимые для представления информации об элементах образа программы (последовательностях инструкций и данных), расположенных в памяти
 - a. Для каждой инструкции - имя мнемоники и набор операндов в терминах данной ВМ
 - b. Для элемента данных - соответствующее литеральное значение или размер экземпляра типа данных в байтах
2. Реализовать модуль, формирующий образ программы в линейном коде для данного набора подпрограмм
 - a. Программный интерфейс модуля принимает на вход структуру данных, содержащую графы потока управления и информацию о локальных переменных и сигнатурах для набора подпрограмм, разработанную в задании 2 (п. 1.a, п. 2.b)
 - b. В результате работы порождается структура данных, разработанная в п. 1, содержащая

описание образа программы в памяти: набор именованных элементов данных и набор

именованных фрагментов линейного кода, представляющих собой алгоритмы подпрограмм

с. Для каждой подпрограммы посредством обхода узлов графа потока управления в порядке

топологической сортировки (начиная с узла, являющегося первым базовым блоком алгоритма

подпрограммы), сформировать набор именованных групп инструкций, включая пролог и

эпилог подпрограммы (формирующие и разрушающие локальное состояние подпрограммы)

d. Для каждого базового блока в составе графа потока управления сформировать группу

инструкций, соответствующих операциям в составе дерева операций

e. Использовать имена групп инструкций для формирования инструкций перехода между

блоками инструкций, соответствующих узлам графа потока управления, в соответствии с

дугами в нём

3. Доработать тестовую программу, разработанную в задании 2 для демонстрации работоспособности

созданного модуля

a. Добавить поддержку аргумента командной строки для имени выходного файла, вывод

информации о графах потока управления сделать опциональных

b. Использовать модуль, разработанный в п. 2 для формирования образа программы на основе

информации, собранной в результате работы модуля, созданного в задании 2 (п. 2.b)

с. Для сформированного образа программы в линейном коде вывести в выходной файл

ассемблерный листинг, содержащий мнемоническое представление инструкций и данных, как

они описаны в структурах данных (п. 1), построенных разработанным модулем (пп. 2.с-е)

d. Проверить корректность решения посредством сборки сгенерированного листинга и запуска

полученного бинарного модуля на эмуляторе ВМ (см. подготовка п. 1.с или п. 2.е)

4. Результаты тестирования представить в виде отчета, в который включить:

a. В части 3 привести описание разработанных структур данных

b. В части 4 описать программный интерфейс и особенности реализации разработанного модуля

с. В части 5 привести примеры исходных текстов, соответствующие ассемблерные листинги и

примера вывода запущенных тестовых программ

Выбран вариант с виртуальной машиной.

Вариант 3

source: sourceItem*;

typeRef: {

|builtin: 'bool'|'byte'|'int'|'uint'|'long'|'ulong'|'char'|'string';

|custom: identifier;

|array: typeRef '(' (',')* ')';

};

```

funcSignature: identifier '(' list<argDef> ')' ('as' typeRef)? {
argDef: identifier ('as' typeRef)?;
};
sourceItem: {
|funcDef: 'function' funcSignature statement* 'end' 'function';
};
statement: {
|var: 'dim' list<identifier> 'as' typeRef; // for static typing
|if: 'if' expr 'then' statement* ('else' statement*)? 'end' 'if';
|while: 'while' expr statement* 'wend';
|do: 'do' statement* 'loop' ('while'|'until') expr;
|break: 'break';
|expression: expr ';';
};
expr: { // присваивание через '='
|binary: expr binOp expr; // где binOp - символ бинарного оператора
|unary: unOp expr; // где unOp - символ унарного оператора
|braces: '(' expr ')';
|callOrIndexer: expr '(' list<expr> ')';
|place: identifier;
|literal: bool|str|char|hex|bits|dec;
};

```

Статическая типизация, стековый формат инструкций ВМ, банки памяти:
code, constants, data.

3. Описание работы

Третья лабораторная работа завершает цепочку трансляции: от исходного текста программы к исполняемому коду виртуальной машины. На основе графов потока управления, построенных во второй работе, система

генерирует линейный код в виде ассемблерного листинга, который можно собрать и запустить на целевой архитектуре.

4. Аспекты реализации

4.1. Компоненты системы

FunExecution + CFG + OperationTree + архитектура BM -> listing.c -> tryPrintOperationTree() -> tryPrintNode() -> placeLabels() -> printListing() -> listing.osm

ListingNode - ассоциация узла с меткой, каждому узлу графа потока назначается уникальная метка для формирования инструкций условных и безусловных переходов.

```
struct ListingNode {
    CFGNode *node;      // Указатель на узел CFG
    char *label;        // Метка для переходов (label0, label1, ...)
    int checked;        // Счётчик посещений для контроля потока
};
```

ValuePlaceAssociation - размещение переменных, отслеживает расположение каждой локальной переменной и аргумента в стеке относительно базового указателя BP.

```
struct ValuePlaceAssociation {
    char *name;         // Имя переменной
    char *type;         // Тип данных (int, char, bool, array)
    int shiftPosition;  // Смещение от BP в словах (1 слово = 2 байта)
};
```

4.2. Модель регистров

Регистры общего назначения:

SP (Stack Pointer) [16 бит] - указатель на вершину стека
IP (Instruction Pointer) [16 бит] - адрес следующей инструкции
BP (Base Pointer) [16 бит] - базовый адрес фрейма стека

Порты ввода-вывода:

IN_PORT [8 бит] - порт ввода (скрытый регистр)
OUT_PORT [8 бит] - порт вывода (скрытый регистр)

4.3. Модель памяти

```
struct Memory {  
    code[0x0000..0xFFFF]    // Сегмент кода  
    databank[0x0000..0xFFFF] // Сегмент данных  
    constants[0x0000..0xFFFF] // Сегмент констант  
}
```

Стек растёт от высших адресов к низшим, аргументы передаются в стеке слева направо, возвращаемое значение размещается на вершине стека перед RET, вызываемая функция отвечает за восстановление SP.

4.4. Инструкции

Мнемоника PUSH получает аргумент - значение (64 бит), кладёт значение на стек.

Мнемоника POP удаляет верхний элемент стека.

Мнемоника LOAD_BP получает аргумент - смещение (16 бит) загружает данные из фрейма в стек.

Мнемоника SAVE_BP получает аргумент - смещение (16 бит) сохраняет данные из стека во фрейм.

Пример:

Загрузить локальную переменную по смещению 4 от BP

LOAD_BP 4

Эффект: $SP -= 16$; $code[SP] = code[BP + 4*8]$; $code[SP+8] = code[BP + 4*8+8]$;

Мнемоники `LOAD_IN`, `SAVE_OUT` читают из порта вывода на стек и записывают со стека в порт вывода.

Формат данных на стеке: каждое значение занимает 16 бит (2 ячейки памяти).

Первый байт - тип (1=int, 3=char, другие значения для других типов), второй байт - значение.

Также представлены мнемоники для различных логических и арифметических операций: `SUM`, `SUB`, `MUL`, `DIV`, `MOD`, `OR` и так далее.

Пример вычисления $1+1$

`PUSH 1` ; тип: int

`PUSH a` ; значение a

`PUSH 1` ; тип: int

`PUSH b` ; значение b

`SUM` ; вычислить сумму

Представлены мнемоники для сравнений: `LT`, `LTE`, `GT`, `GTE`, `EQ`, `NEQ`.

Представлены мнемоники управления потоком: `JZ`, `JNZ`, `JMP` и так далее.

Пример условного ветвления:

; Если $x < 0$, то $y = -x$, иначе $y = x$

`LOAD_BP 0` ; загрузить x

`PUSH 1` ; тип: int

`PUSH 0` ; ноль для сравнения

`LT` ; $x < 0$?

`JZ else_label` ; если false, перейти на else

; then блок

`PUSH 0` ; ноль

`LOAD_BP 0` ; x

`SUB` ; $0 - x = -x$

```
JMP end_label
else_label:
; else блок
LOAD_BP 0    ; x
end_label:
; объединение потоков
SAVE_BP 1    ; сохранить результат в y
```

4.5. Алгоритм генерации листинга

Алгоритм будет представлен в виде нескольких кратких блоков псевдокода для упрощения

1 - Расстановка меток (placeLabels)

INPUT: Array<FunExecution> функции

OUTPUT: Каждый узел CFG содержит ListingNode с уникальной меткой

ALGORITHM:

labelCounter = 0

FOR EACH функция DO

FOR EACH узел в CFG (DFS) DO

IF узел посещён впервые THEN

createListingNode(узел)

END IF

IF узел имеет условный переход THEN

// Условный переход требует метку

IF узел.conditionally не имеет метку THEN

labelCounter++

узел.conditionally.label = "label" + labelCounter

END IF

END IF

IF узел.definitely уже посещён AND не равен NULL THEN

// Обратный переход (цикл) - нужна метка

IF узел.definitely не имеет метку THEN

labelCounter++

узел.definitely.label = "label" + labelCounter

END IF

END IF

END FOR

END FOR

2 - Размещение переменных (addArgumentPlace, addValuePlace)

INPUT: Операционное дерево узла CFG

OUTPUT: Array<ValuePlaceAssociation> с информацией о переменных

ALGORITHM:

FUNCTION addArgumentPlace(name, type):

// Аргументы размещаются перед return address и BP

// Первый аргумент: BP - 2

// Второй аргумент: BP - 4

// и т.д.

FOR EACH существующий аргумент DO

аргумент.shiftPosition -= 2

END FOR

создать новый ValuePlaceAssociation

newArg.shiftPosition = -2

RETURN newArg

FUNCTION addValuePlace(name, type):

// Локальные переменные размещаются после return address

// Первая переменная: BP + 2

// Вторая переменная: BP + 4

IF существует предыдущая локальная переменная THEN

newVar.shiftPosition = prev.shiftPosition + 2

ELSE

newVar.shiftPosition = 2

END IF

RETURN newVar

3 - Генерация инструкций (printListing, tryPrintNode)

INPUT: CFG с расставленными метками и размещёнными переменными

OUTPUT: Мнемонический листинг

ALGORITHM printListing:

WRITE "[section code]"

WRITE "CALL main"

WRITE "POP"

WRITE "HLT"

FOR EACH функция DO

argumentNumber = 0

valuePlaces = []

WRITE функция.name + ":"

```
WRITE "PUSH 0"      // Инициализация типа для return value  
WRITE "PUSH 0"
```

```
    processNode(функция.nodes, valuePlaces, argumentNumber)  
END FOR
```

```
ALGORITHM processNode(node, valuePlaces, argumentNumber):
```

```
    IF node == NULL THEN RETURN END IF
```

```
    IF node имеет метку THEN
```

```
        WRITE node.label + ":"
```

```
    END IF
```

```
    node.checked++
```

```
    IF node содержит операционное дерево THEN
```

```
        processOperationTree(node.operationTree)
```

```
    IF node имеет условный переход THEN
```

```
        WRITE "JNZ " + node.conditionally.label
```

```
    END IF
```

```
END IF
```

```
IF node.definitely == NULL THEN
```

```
    // Конец функции
```

```
    WRITE "SAVE_BP 1"
```

```
    WRITE "RET " + argumentNumber
```

```
ELSE IF node.definitely уже посещён THEN
```

```
    // Переход на ранее посещённый узел (цикл)
```

```
WRITE "JMP " + node.definitely.label  
ELSE  
    // Продолжение потока управления  
    processNode(node.definitely, ...)  
END IF
```

```
IF node.conditionally нужна обработка THEN  
    processNode(node.conditionally, ...)  
END IF
```

4 - Преобразование операционных деревьев

ALGORITHM processOperationTree(opTree):

```
SWITCH opTree.type OF
```

```
CASE "ARG":
```

```
    argumentNumber++
```

```
    addArgumentPlace(opTree.childNodes[1].value,  
                    opTree.childNodes[0].value)
```

```
CASE "AS" (присваивание):
```

```
    addValuePlace(opTree.childNodes[1].value,  
                opTree.childNodes[0].value)
```

```
    WRITE "PUSH 0"
```

```
    WRITE "PUSH 0"
```

```
CASE "CONST":
```

```
    type = opTree.childNodes[0].value
```

```
    value = opTree.childNodes[1].value
```

```

IF type == "int" THEN
    WRITE "PUSH 1"          // Тип: int
    WRITE "PUSH " + value
ELSE IF type == "char" THEN
    WRITE "PUSH 3"          // Тип: char
    WRITE "PUSH " + ord(value)
ELSE IF type == "bool" THEN
    WRITE "PUSH 1"          // Тип: bool (как int)
    WRITE "PUSH " + (value == "true" ? 1 : 0)
END IF

```

```

CASE "SET" (присваивание переменной):
    processOperationTree(opTree.childNodes[1])
    varPlace = findVariable(opTree.childNodes[0].value)
    WRITE "SAVE_BP " + varPlace.shiftPosition

```

```

CASE "READ" (чтение переменной):
    varPlace = findVariable(opTree.childNodes[0].value)
    WRITE "LOAD_BP " + varPlace.shiftPosition

```

```

CASE "EQUALITY" (==):
    processOperationTree(opTree.childNodes[0])
    processOperationTree(opTree.childNodes[1])
    WRITE "EQ"

```

```

CASE "LESSTHAN" (<):
    processOperationTree(opTree.childNodes[0])
    processOperationTree(opTree.childNodes[1])
    WRITE "LT"

```


CASE "SUM" (+):

processOperationTree(opTree.childNodes[0])

processOperationTree(opTree.childNodes[1])

WRITE "SUM"

// ... остальные операции (SUB, MUL, DIV, MOD, BIT_AND, OR)

CASE "EXECUTE" (вызов функции):

funcName = opTree.childNodes[0].value

IF funcName == "input" THEN

WRITE "LOAD_IN"

ELSE IF funcName == "output" THEN

processOperationTree(opTree.childNodes[1])

WRITE "SAVE_OUT"

ELSE

// Пользовательская функция

FOR i = 1 TO opTree.childrenNumber-1 DO

processOperationTree(opTree.childNodes[i])

END FOR

WRITE "CALL " + funcName

END IF

END SWITCH

Результаты

Были созданы такие артефакты как listing.h - определения структур

ListingNode и ValuePlaceAssociation, listing.c - реализация генератора кода,

arch.pdsl - описание архитектуры ВМ, модуль placeLabels - расстановка меток, модуль printListing - основной цикл генерации, модуль tryPrintNode - обход CFG и вывод инструкций, модуль tryPrintOperationTree - преобразование операций.

Пример для исходного текста:

```
function read_num() as int
  dim in as int
  dim res as int
  res = 0;
  while true
    in = input();
    if in == 13 then
      break
    end if
    res = res * 10 + (in - 48);
  wend
  res;
end function
```

Листинг:

[section code]

CALL main

POP

HLT

read_num:

PUSH 0

PUSH 0

PUSH 0

PUSH 0

PUSH 0
PUSH 0
PUSH 1
PUSH 0
SAVE_BP 5
label2:
PUSH 1
PUSH 1
JNZ label1
label4:
LOAD_BP 5
SAVE_BP 1
RET 0
label1:
LOAD_IN
SAVE_BP 3
LOAD_BP 3
PUSH 1
PUSH 13
EQ
JNZ label3
LOAD_BP 5
PUSH 1
PUSH 10
MUL
LOAD_BP 3
PUSH 1
PUSH 48
SUB

```
SUM
SAVE_BP 5
JMP label2
label3:
JMP label4
```

Выводы

В третьей лабораторной работе успешно реализована заключительная стадия цепочки трансляции - генерация линейного машинного кода на основе графов потока управления, построенных во второй работе. Разработан модуль listing, который преобразует иерархическое представление CFG в последовательность мнемонических инструкций целевой виртуальной машины, полностью описанной в нотации PDSL с 31 инструкцией, включая операции данных (PUSH, POP, LOAD_BP, SAVE_BP), арифметические и логические операции (SUM, SUB, MUL, DIV, MOD, AND, OR), сравнения (LT, LTE, GT, GTE, EQ, NEQ) и управления потоком (JZ, JNZ, JMP, CALL, RET). Реализованы четыре ключевые фазы генерации: автоматическая расстановка меток для всех точек ветвления и циклических переходов (placeLabels), правильное размещение локальных переменных и аргументов на стеке с использованием базового указателя BP (addArgumentPlace, addValuePlace), обход графа потока управления с эмиссией инструкций (tryPrintNode) и преобразование операционных деревьев в последовательности инструкций (tryPrintOperationTreeNode).